

# Introdução à Programação em Python



Fundamentos do Python

---

Ronaldo Verçosa

# O que é programação?



Linguagem de  
Programação

É uma linguagem que comunica instruções a um computador.

Pensem nela como uma "tradutora" entre o que queremos e o que a máquina entende.

O que podemos fazer?



Realizar cálculos



Analisar textos



Ler e gravar arquivos



Exibir imagens na tela



Aceitar entradas do usuário

# Automação: por que programar?



Automação é qualquer tecnologia que remove a interação humana de um processo.  
Scripts são escritos para automatizar tarefas repetitivas e simplificar o trabalho.

1

Quanto automatizar  
para atingir  
seus objetivos?

2

Identifique os  
processos adequados  
para automatizar

3

Encontre o  
equilíbrio entre  
excesso e falta

4

Não automatize  
um processo  
ruim!

---

# Como um software é escrito?



## Arquivos de Texto

O código-fonte é escrito em arquivos de texto simples (.py, .js, .c)

Editores populares:

VS Code, Sublime Text, Vim, nano, Notepad++



## Linguagens

Cada linguagem tem sua gramática, sintaxe e comunidade

Exemplos:

Python, JavaScript, C#, C/C++, Java

**O Microsoft Word NÃO é um editor de texto para código (.docx)!**

# Ambiente de Desenvolvimento Integrado (IDE)

*O IDE é como um "corretor ortográfico" para o seu código!*

## Detecta erros

Identifica palavras escritas incorretamente e sintaxe errada

## Sugere correções

A maioria dos IDEs sugere como corrigir o problema encontrado

## Foco na linguagem

Alguns IDEs são especializados, como o PyCharm para Python

# Compiladores vs Intérpretes

Ambos traduzem código de alto nível em código de máquina (0s e 1s)

## Compiladas

Traduzem TODO o código de uma vez, antes de executar.

Exemplos: C/C++, GoLang, Basic

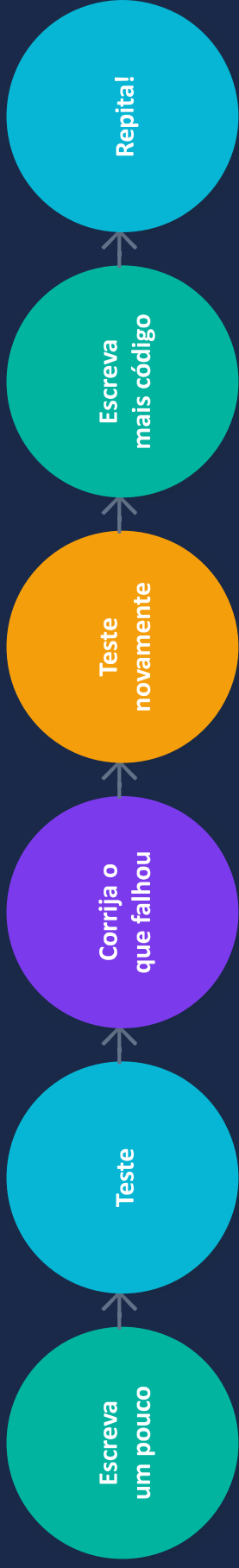
## Interpretadas

Traduzem linha por linha, enquanto o código está rodando.

Exemplos: Python, Ruby, JavaScript

# O ciclo de desenvolvimento

*Software é escrito de forma iterativa — passo a passo!*



Execute frequentemente → Encontre erros cedo → Corrija antes que cresçam → Continue evoluindo!

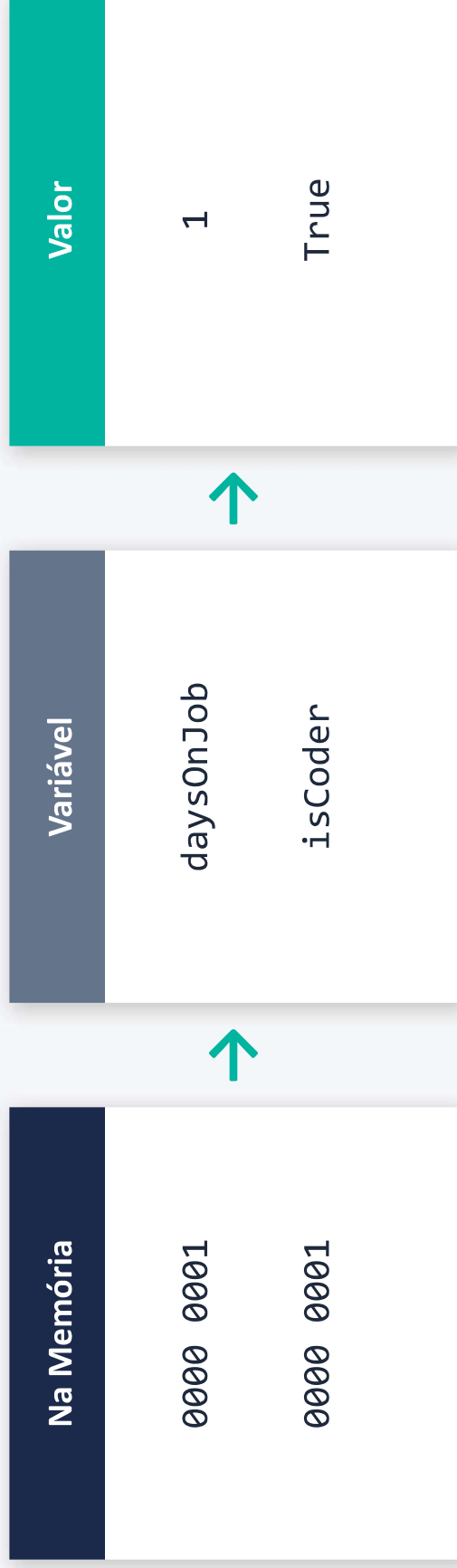
# Tipos de Dados

*Um tipo de dados informa ao computador como interpretar um valor*

|                                                                            |                 |
|----------------------------------------------------------------------------|-----------------|
| 45                                                                         | Inteiro (int)   |
| 1.02                                                                       | Float (decimal) |
| True / False                                                               | Booleano (bool) |
| "Olá mundo"                                                                | String (str)    |
| "45"                                                                       | String (str)    |
| Atenção: "45" com aspas é String, não número! O tipo muda a interpretação. |                 |

# O que é uma variável?

Uma variável é como uma "caixinha" que guarda um valor na memória do computador.  
O nome da variável ajuda a lembrar o que ela armazena.



```
Python: isCoder = True    daysOnJob = 1    nome = "Maria"
```

# Funções

*Funções são blocos de código reutilizáveis que fazem uma tarefa específica*

Fazem algo útil

```
clearScreen()
```

Retornam valores

```
pi = calculatePi()
```

Aceitam entradas

```
showValue(area)
```

Aceitam múltiplas entradas

```
fly(lat, lon, spd)
```

# Controle de Fluxo

## Condicionais (if/elif/else)

Permitem escolher caminhos diferentes no código

```
if guess > number:
    print("Muito alto!")
elif guess < number:
    print("Muito baixo!")
else:
    print("Acertou!")
```

## Loops (for/while)

Repetem ações para cada item de uma coleção

```
names = ["wei", "nikki"]

for name in names:
    newName = name.upper()
    print(newName)

# Saída: WEI, NIKKI
```

# Controle de Versão com Git

Controle de versão é um software que rastreia todas as mudanças no seu código.

Pense nele como um "histórico infinito" do seu projeto!

## Acompanhamento

Rastreia todos os erros e mudanças

## Segurança

Backup na nuvem, menos risco de perda

## Colaboração

Várias pessoas trabalham no mesmo projeto

Git = ferramenta local

GitHub = hospedagem na nuvem

```
git init → git add arquivo → git commit -m "mensagem" → git push
```